

Classification & Regression Model Benchmark

세 가지 Kaggle 과제(이진분류 2 · 회귀 1)에서 각 폴더의 전처리 방식을 준수한 채 LogisticRegression · Ridge · RandomForest · GBM · XGBoost · LightGBM 을 학습하고 분류 5지표 / RMSLE 로 비교한 통합 보고서

Santander Customer Satisfaction

Credit Card Fraud Detection

Mercari Price Suggestion

scikit-learn · XGBoost · LightGBM

3

분석 주제 (분류 2 · 회귀 1)

13

학습·평가한 모델 수

5 + 1

분류 지표 / 회귀 지표(RMSLE)

1.85M

총 데이터 행 수 (3주제 합)

01 서론

연구 배경 · 과제 목적 · 평가 지표

1.1 연구 배경 및 필요성

실무 데이터 과제는 (1) 심한 클래스 불균형이 있는 **이진분류**와 (2) 값의 범위가 넓고 오른쪽으로 치우친 **회귀**로 크게 나뉜다. 두 유형 모두 "어떤 전처리를 하고 어떤 모델을 쓰는가"에 따라 성능이 크게 달라지므로, 서로 다른 성격의 대표 과제 3개에 대해 동일한 파이프라인 관점(데이터 → 전처리 → 다중 모델 학습 → 지표 비교)을 적용해 모델군의 상대적 강약을 확인할 필요가 있다.

1.2 과제 목적

- 세 과제(Santander 고객 만족도 · Credit Card 사기탐지 · Mercari 가격예측) 각각에 대해, 해당 폴더의 노트북에 정리된 **전처리 방식**을 그대로 준수한다.
- 분류 과제에는 `LogisticRegression` · `RandomForest` · `GBM` · `XGBoost` · `LightGBM` 5종을, 회귀 과제에는 `Ridge` · `XGBoost` · `LightGBM` 3종을 동일한 train/test 분할로 학습한다.
- 분류는 **Accuracy** · **Precision** · **Recall** · **F1** · **ROC-AUC**, 회귀는 **RMSLE**를 산출해 정리한다.

1.3 평가 지표

지표	정의 / 의미
Accuracy	전체 예측 중 맞춘 비율. 불균형 데이터에서는 다수 클래스만 찍어도 높게 나와 단독 해석은 주의.
Precision	양성으로 예측한 것 중 실제 양성 비율(오탐 억제 관점).
Recall	실제 양성 중 잡아낸 비율(미탐 억제 관점). 사기·불만족 탐지에서 특히 중요.
F1	Precision과 Recall의 조화평균. 임계값 0.5 기준의 균형 성능.
ROC-AUC	임계값과 무관하게 양성/음성 순위를 얼마나 잘 매기는지. 불균형에서 가장 안정적인 비교 지표.
RMSLE	$\sqrt{\text{mean}((\log(1+\hat{y}) - \log(1+y))^2)}$. 상대오차 기준이라 가격대가 넓은 회귀에 적합. 낮을수록 좋음.

※ 모든 분류 지표는 임계값 0.5 기준. 실행환경: Anaconda `ml-dev` (scikit-learn 1.9 · XGBoost 3.2 · LightGBM 4.7).

02 주제 1 — Santander Customer Satisfaction

이진분류 · 고객 불만족 예측 · 심한 클래스 불균형

2.1 데이터셋 소개

76,020

학습 행 수

371

원본 컬럼 수

145

전처리 후 피처 수

3.96%

불만족(TARGET=1) 비율

익명화된 금융 피처(`var3`, `var15`, `saldo_*` 등) 약 370개로 고객의 불만족 여부(`TARGET`=1)를 예측한다. 양성 비율이 4% 미만인 전형적 불균형 데이터이며, 분산이 0에 가까운 컬럼·완전 중복 컬럼이 다수이고 `var3`에는 결측치를 뜻하는 `-999999`가 섞여 있다.

2.2 전처리 (폴더 방식 준수)

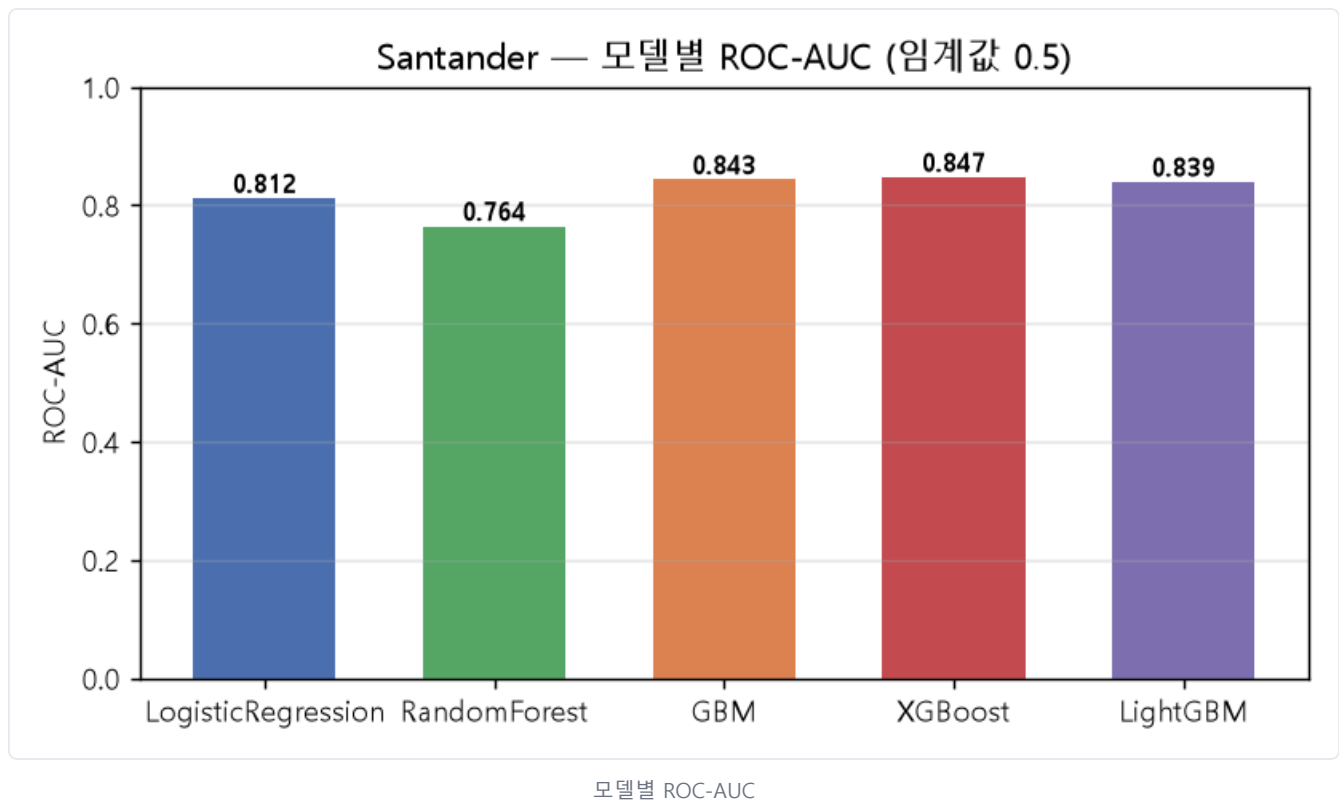
01_Santander_Logistic_Regression.ipynb의 최종 채택 `preprocess()`를 그대로 사용한다.

- `var3`의 `-999999` → `2` (최빈값)로 치환
- `ID`, `TARGET` 분리 후 **완전 중복 컬럼·표준편차 0 컬럼** 제거
- **희소 컬럼**(값의 99% 이상이 0) 제거 → 학습 속도 크게 개선
- `var38` `log1p` 변환, `var15` 파생 (`var15_below_23`, `var15_bin`)
- → 최종 **145개 피처**, `StandardScaler` 표준화, `train_test_split(test_size=0.2, random_state=42, stratify=y)`

2.3 모델별 성능 (임계값 0.5)

모델	Accuracy	Precision	Recall	F1	ROC-AUC	학습(s)
LogisticRegression	0.7459	0.1068	0.7359	0.1866	0.8116	14.8
RandomForest	0.9509	0.1571	0.0548	0.0813	0.7643	2.8
GBM	0.9601	0.2222	0.0033	0.0065	0.8430	24.1
XGBoost	0.9605	0.5714	0.0066	0.0131	0.8472	1.6
LightGBM	0.9600	0.2000	0.0033	0.0065	0.8392	1.2

※ 강조 행 = ROC-AUC 최고 모델. 다만 이 데이터는 임계값 0.5·무가중 상태라 부스팅 모델의 Recall/F1은 사실상 무의미하며, 실사용 관점의 해석은 2.4를 참조.



2.4 해석

- **LogisticRegression**만 `class_weight='balanced'` 를 적용해 Recall 0.736을 확보했으나, 선형 결정경계 한계로 ROC-AUC는 0.8116에 머문다.
- 트리 부스팅 계열은 ROC-AUC에서 우세하다 — **XGBoost 0.8472**, GBM 0.8430, LightGBM 0.8392. 즉 순위 매김 능력 자체는 비선형 모델이 더 낫다.
- 다만 부스팅 모델은 무가중·임계값 0.5라 다수 클래스로 쏠려 Recall이 0에 가깝다(F1도 붕괴). 실사용하려면 `class_weight / scale_pos_weight` 또는 임계값 튜닝이 필수다.
- RandomForest(기본값)는 ROC-AUC 0.7643로 가장 낮아, 이 데이터에서는 부스팅 대비 이점이 없다.

03 주제 2 — Credit Card Fraud Detection

이진분류 · 신용카드 사기 탐지 · 극단적 클래스 불균형(0.17%)

3.1 데이터셋 소개

284,807

원본 행 수

31

원본 컬럼 수

284,803

이상치 제거 후 행 수

0.171%

사기(Class=1) 비율

시간(`Time`), 거래금액(`Amount`), 그리고 익명화·PCA 처리된 `V1~V28`로 사기 거래(`Class`=1)를 탐지한다. 사기 비율이 0.17%로 Santander보다도 훨씬 극단적인 불균형이다.

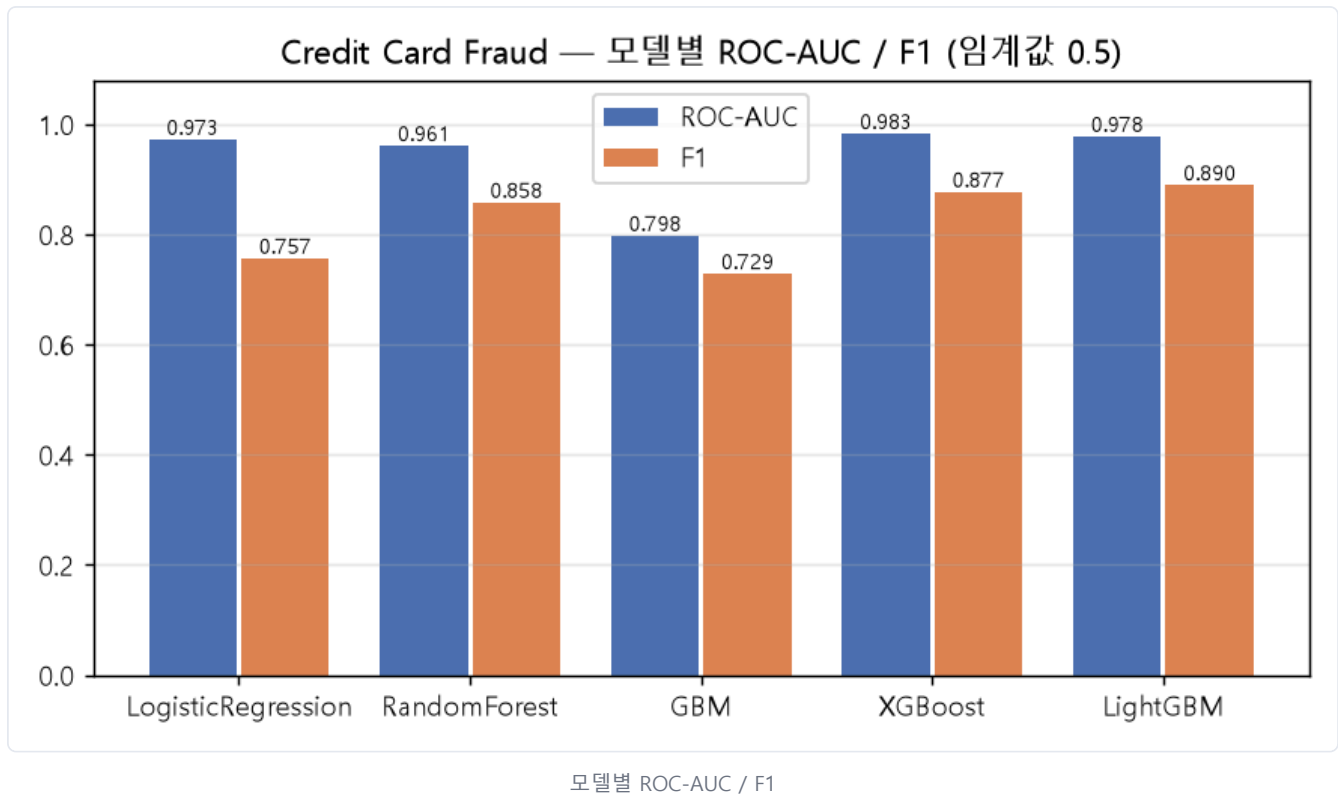
3.2 전처리 (폴더 방식 준수)

`02_Credit_card_fraud_detection_Random_Forest.ipynb`의 최종 `get_preprocessed_df()`를 그대로 사용한다.

- `Time` 컬럼 삭제
- `Amount` → `log1p` 변환하여 `Amount_Scaled`로 대체 (큰 값은 많이, 작은 값은 적게 눌러 분포를 펴)
- 사기 데이터 기준 `V14`의 IQR 이상치(`weight=1.5`) 4건 제거
- 284,803 × 29 행렬, `train_test_split(test_size=0.3, random_state=0, stratify=y)`

3.3 모델별 성능 (임계값 0.5)

모델	Accuracy	Precision	Recall	F1	ROC-AUC	학습(s)
LogisticRegression	0.9993	0.8673	0.6712	0.7568	0.9725	0.5
RandomForest	0.9996	0.9426	0.7877	0.8582	0.9611	26.2
GBM	0.9992	0.8393	0.6438	0.7287	0.7978	209.9
XGBoost	0.9996	0.9593	0.8082	0.8773	0.9830	2.9
LightGBM	0.9996	0.9603	0.8288	0.8897	0.9780	6.4



3.4 해석

- **XGBoost**가 ROC-AUC **0.9830**로 최고, **LightGBM**이 F1 **0.8897**(Recall 0.829)로 최고다. 두 부스팅 구현체가 극단 불균형에서도 사기 순위를 잘 매긴다.
- **sklearn GBM**은 ROC-AUC 0.7978로 크게 뒤처지고 학습도 210초로 가장 느리다 — 기본 100트리·deviance 손실이 0.17% 불균형에 취약함을 보여준다.
- RandomForest는 F1 0.8582로 견조하나 ROC-AUC 0.9611는 부스팅보다 낮다. LogisticRegression도 ROC-AUC 0.9725로 준수하지만 Recall 0.671은 트리 앙상블보다 낮다.
- 이 과제는 Santander와 달리 임계값 0.5에서도 F1·Recall이 의미 있게 나오는데, 이는 사기/정상이 피쳐 공간에서 비교적 잘 분리되기 때문이다(V14/V12/V17 등).

04 주제 3 — Mercari Price Suggestion

회귀 · 상품 판매가 예측 · 초고차원 희소 텍스트 피처

4.1 데이터셋 소개

1,482,535

학습 행 수

8

원본 컬럼 수

105,812

벡터화 후 피처 차원

RMSLE

평가 지표

상품명(`name`), 설명(`item_description`), 브랜드, 카테고리, 상태, 배송 정보로 판매가(`price`)를 예측한다. 가격은 오른쪽으로 크게 치우친 분포(왜도 ≈ 11.4)라 학습 타겟으로는 `log1p(price)`를 쓴다.

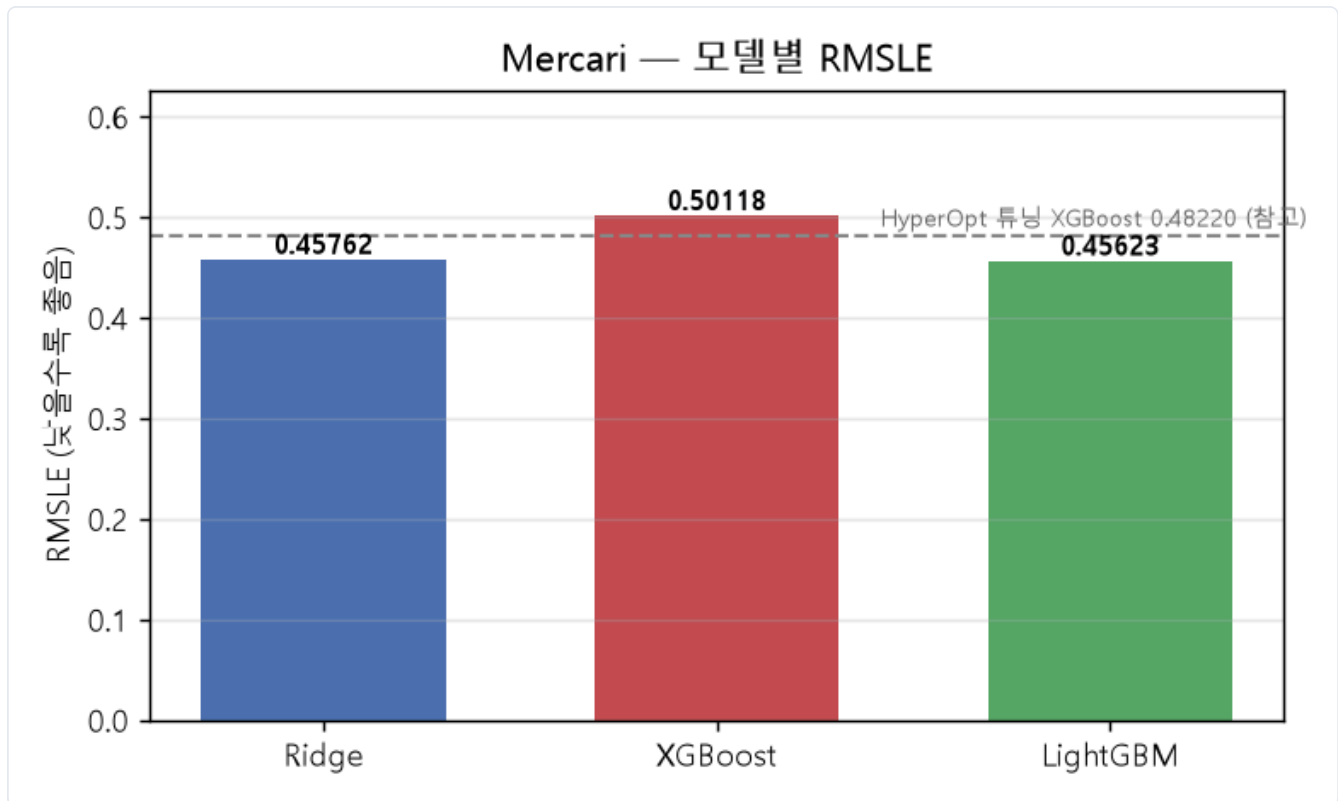
4.2 전처리 (폴더 방식 준수)

Mercari price suggestion XGB.ipynb의 벡터화 파이프라인을 그대로 사용한다.

- 타겟 `log1p` 변환, 결측은 `'Other_Null'`로 대체
- `category_name`을 `/` 기준 대/중/소 3개 컬럼으로 분할
- `name` → TF-IDF (1,2)-gram / 50,000, `item_description` → TF-IDF (1,3)-gram / 50,000
- 브랜드·상태·배송·카테고리 3종 → `LabelBinarizer` 원-핫 (희소)
- `scipy.sparse.hstack`으로 결합, 최종 $1,482,535 \times 105,812$ 희소행렬(CSR), `train_test_split(test_size=0.2, random_state=156)`

4.3 모델별 성능 (RMSLE, 낮을수록 좋음)

모델	RMSLE	학습(s)
Ridge	0.45762	28.0
XGBoost	0.50118	503.8
LightGBM	0.45623	832.3



모델별 RMSLE (점선: HyperOpt 튜닝 XGBoost 참고치)

HyperOpt 튜닝 XGBoost RMSLE 0.48220 (기존 노트북 결과) 은 별도 참고치. 본 표는 튜닝 없이 노트북 기본 파라미터로 재학습한 결과다.

4.4 해석

- **LightGBM**이 RMSLE **0.45623**로 근소하게 최저지만, **Ridge** (0.45762)와의 차이는 약 0.00139로 사실상 동률이다. 반면 기본 파라미터 **XGBoost**는 0.50118로 뚜렷하게 뒤쳐진다.
- 결정적인 차이는 **학습 비용**이다 — Ridge 28s vs LightGBM 832s(30배), XGBoost 504s. LightGBM은 num_leaves=125·learning_rate=0.5로 공격적으로 맞춰야 겨우 Ridge와 동률이 되고, 평범한 XGBoost 설정은 한참 못 미친다.
- TF-IDF/원-핫으로 만든 105,812차원 희소행렬은 각 단어(피처)와 가격의 관계가 대체로 선형에 가깝고 한 행에서 0이 아닌 값이 극소수다. 이런 초고차원 희소 데이터에서는 "피처마다 계수 하나"를 동시에 푸는 규제 선형모델이 **비용 대비 가장 효율적**이다.
- 참고로 XGBoost는 HyperOpt 30회 튜닝(≈1시간 53분) 후에도 RMSLE 0.48220으로 Ridge·LightGBM에 미치지 못했다.

05 종합 결과 및 결론

주제별 최고 성능 · 모델군 비교 · 결론

5.1 주제별 최고 성능 요약

주제	유형	대표 지표	최고 모델	값
Santander Customer Satisfaction	이진분류	ROC-AUC	XGBoost	0.8472
Credit Card Fraud Detection	이진분류	ROC-AUC	XGBoost	0.9830
Credit Card Fraud Detection	이진분류	F1	LightGBM	0.8897
Mercari Price Suggestion	회귀	RMSLE ↓	LightGBM	0.45623

5.2 모델군 비교

모델군	분류에서	회귀(회소 텍스트)에서
선형 (LogReg / Ridge)	ROC-AUC는 부스팅보다 낮지만, <code>class_weight='balanced'</code> 로 임계값 0.5에서도 Recall 확보. 해석·속도 이점.	LightGBM과 사실상 동률 (RMSLE 0.4576 vs 0.4562)이면서 학습은 30배 빠름 — 비용 대비 최적 .
배깅 (RandomForest)	Credit Card에서는 F1 견조, Santander에서는 최하위. 기본값으로는 불균형 대응이 약함.	차원·메모리 제약으로 사실상 학습 불가(본 보고서 제외).
부스팅 (GBM / XGBoost / LightGBM)	XGBoost-LightGBM이 ROC-AUC·F1 최상위. 단 sklearn GBM 기본설정 은 극단 불균형에 취약. 임계값·가중치 보정 없으면 Recall 붕괴.	LightGBM은 공격적 설정으로 Ridge와 겨우 동률, 기본 XGBoost는 뒤처짐. 학습 비용만 수십 배.

5.3 결론

- 분류** — 순위 품질(ROC-AUC)만 보면 XGBoost/LightGBM이 두 과제 모두에서 최상위다. 그러나 불균형이 심할수록 (a) sklearn GBM 기본설정은 신뢰하기 어렵고, (b) 임계값 0.5·무가중 상태에서는 Recall/F1이 무너지므로 **class_weight·scale_pos_weight·임계값 튜닝이 사실상 필수**다. LogisticRegression은 이 보정을 기본 옵션 하나로 처리할 수 있어 베이스라인으로 여전히 유효하다.
- 회귀** — 10만 차원 회소 텍스트 피처에서는 규제 선형모델 **Ridge**(RMSLE 0.45762)와 공격적으로 튜닝한 **LightGBM**(0.45623)이 사실상 동률이며, 학습 비용은 Ridge가 30배 저렴하다. 기본 설정 XGBoost(0.50118)와 HyperOpt 튜닝 XGBoost(0.4822)는 모두 이에 못 미친다 → 비용 대비 규제 선형모델이 가장 합리적인 선택이다.
- 공통** — 세 과제 모두 왜곡된 수치 피처/타겟의 `log1p` 변환 (`var38` · `Amount` · `price`)이 안정적 학습의 핵심이었다.

5.4 실행환경

Anaconda 가상환경 `ml-dev` · Python 3 · scikit-learn 1.9.0 · XGBoost 3.2.0 · LightGBM 4.7.0 · 동일 `random_state` 고정(분류 42/0, 회귀 156). 분류 지표는 임계값 0.5 기준.